

EE5770: Course Project

LDPC Codes vs. Terminated Convolutional Codes

5G NR PDSCH Chain with OFDM over an L -Tap Fading Channel

Overview and Objective

The Physical Downlink Shared Channel (PDSCH) is the primary data-bearing channel in 5G NR, carrying user-plane traffic from the base station to the UE. 5G NR uses LDPC codes for PDSCH because they offer excellent performance at medium-to-large blocklengths and support highly parallel decoding. Terminated trellis-based convolutional codes, decoded via the Viterbi algorithm, offer a useful baseline because they achieve near-maximum-likelihood decoding and have well-understood complexity.

In this project you will:

1. Implement the complete **5G NR PDSCH transmitter and receiver chain** in software, including an OFDM physical layer over an L -tap frequency-selective Rayleigh fading channel.
2. Plug in two different channel coding schemes — 5G NR LDPC codes and terminated convolutional codes — into the same chain.
3. Compare their **error-rate performance and decoding complexity** under a matched-complexity constraint, with **both ZF and MMSE channel equalizers** evaluated independently.

Central constraint. All coding scheme comparisons must be made at *equal average decoding complexity*. Equalizer comparisons must be made with the *same coding scheme and operating point*, so that the only variable is the equalizer. Results without matched complexity will not receive full credit.

Toolbox restriction — read carefully. All signal processing must be implemented **from scratch**. The use of built-in functions from any of the following MATLAB toolboxes is **strictly forbidden**:

- **Communications Toolbox** — e.g. `comm.LDPCDecoder`, `comm.LDPCDecoder`, `comm.ViterbiDecoder`, `comm.ConvolutionalEncoder`, `comm.OFDMModulator`, `comm.OFDMDemodulator`, `comm.QPSKModulator`, `comm.RectangularQAMModulator`, `comm.AWGNChannel`, `comm.RayleighChannel`, `comm.CRCGenerator`, `comm.CRCDetector`, `comm.ScramblingGenerator`, and all related objects.
- **LTE Toolbox** — e.g. `lteTurboEncode`, `lteTurboDecode`, `lteRateMatcher`, `lteRateRecoverTrblk`, `lteDLSCH`, `lteULSCH`, `lteOFDMModulate`, `lteOFDMDemodulate`, `lteEqualizeMMSE`, `lteEqualizeZF`, and all related functions.
- **5G Toolbox** — e.g. `nrLDPCEncode`, `nrLDPCDecode`, `nrRateMatchLDPC`, `nrRateRecoverLDPC`, `nrDLSCH`, `nrULSCH`, `nrOFDMModulate`, `nrOFDMDemodulate`, `nrPDSCH`, `nrPDSCHDecode`, `nrCRCEncode`, `nrCRCDecode`, `nrCodeBlockSegmentLDPC`,

`nrCodeBlockDesegmentLDPC`, and all related functions.

The following **are permitted**: basic MATLAB built-ins (`fft`, `ifft`, `conv`, `randn`, `rand`, `mod`, `de2bi`, `bi2de`), standard numerical and plotting functions (`plot`, `semilogy`, `mean`, `var`). Any use of a forbidden function, even as a reference check, will result in a **deduction of up to 50% marks** and may be treated as academic misconduct. If you are unsure whether a function is permitted, ask before using it.

System Parameters and Configurability

The entire simulation chain must be **fully parameterised**. No parameter should be hard-coded. The following must be configurable at runtime:

Symbol	Description	Example values
A	Transport block size (bits, before CRC)	1000, 3000, 8000
RNTI	Radio Network Temporary Identifier	any 16-bit value
Cell ID	Physical cell identity (scrambling)	0–1007
E	Rate-matched output length (bits)	2000, 6000, 16000
R	Target code rate	1/3, 1/2, 2/3
M	Modulation order	2 (QPSK), 4 (16-QAM), 6 (64-QAM)
N_{FFT}	IDFT/DFT size	256, 512, 1024
L	Number of channel taps	4, 8, 16
N_{CP}	Cyclic prefix length	$\geq L - 1$

You must report results for **at least three distinct configurations**, chosen to span different code rates, modulation orders, and transport block sizes representative of real PDSCH operation.

Channel Model

AWGN-only simulations are not sufficient. All performance curves must be obtained over a **frequency-selective Rayleigh fading channel** modelled as an L -tap multipath channel. AWGN results may be included as a reference but cannot substitute for fading results.

The discrete-time baseband channel is modelled as an L -tap FIR filter:

$$\tilde{y}[n] = \sum_{\ell=0}^{L-1} h[\ell] \tilde{x}[n - \ell] + w[n],$$

where each tap $h[\ell] \sim \mathcal{CN}(0, 1/L)$ is drawn independently per OFDM symbol (block-fading model) and the tap variances are normalised so that $\sum_{\ell} \mathbb{E}[|h[\ell]|^2] = 1$. The additive noise satisfies $w[n] \sim \mathcal{CN}(0, \sigma_w^2)$ with $\sigma_w^2 = 1/(2 \cdot E_b/N_0 \cdot R_{\text{eff}})$, where R_{eff} is the effective code rate after rate matching and modulation.

The cyclic prefix length must satisfy $N_{\text{CP}} \geq L - 1$ to prevent inter-symbol interference between successive OFDM symbols. Channel taps are assumed to be **known**

perfectly at the receiver (perfect CSI). The channel is redrawn independently for each OFDM symbol.

PDSCH Transmitter Chain

Implement the following processing steps in order. Each step must be a **separate, testable function** in your codebase.

Step 1: Transport Block Generation. Generate a binary payload of length A bits. For simulation, use a uniformly random equiprobable bit sequence.

Step 2: CRC Attachment. Compute a 24-bit CRC over the transport block using the polynomial CRC24A as specified in 3GPP TS 38.212, Section 5.1. Append the 24 CRC bits to produce a sequence of $A + 24$ bits.

Step 3: Code Block Segmentation. If $A + 24$ exceeds the maximum code block size for the selected base graph (8448 bits for BG1, 3840 bits for BG2), segment the bit sequence into multiple code blocks and attach an additional 24-bit CRC24B to each segment, following 3GPP TS 38.212, Section 5.2.2. Each code block is encoded independently. For the convolutional code baseline, apply a simple fixed-size segmentation into blocks of at most 6144 bits.

Step 4: Channel Coding. Encode each code block using one of the two schemes (see Section 5). The coded output of all blocks is concatenated before rate matching.

Step 5: Rate Matching. Puncture, repeat, or shorten the concatenated coded bits to the target output length E , following 3GPP TS 38.212, Section 5.4 for LDPC and a configurable puncturing pattern for convolutional codes.

Step 6: Scrambling. XOR the rate-matched bit sequence with a length- E pseudo-random binary sequence generated from the RNTI and the physical cell ID, as specified in 3GPP TS 38.211, Section 7.3.1.1.

Step 7: Modulation. Map groups of M scrambled bits to complex constellation symbols using Gray-coded QPSK ($M = 2$), 16-QAM ($M = 4$), or 64-QAM ($M = 6$), following the bit-to-symbol mapping in 3GPP TS 38.211, Section 5.1. Normalise each constellation to unit average power. This produces E/M complex frequency-domain symbols $S[k]$.

Step 8: Subcarrier Mapping. Map the E/M modulation symbols onto E/M active subcarriers of an N_{FFT} -point OFDM symbol. Assign the DC subcarrier and all guard subcarriers to zero. The result is a frequency-domain vector $\mathbf{S} \in \mathbb{C}^{N_{\text{FFT}}}$.

Step 9: N_{FFT} -Point IDFT (OFDM Modulation). Convert the frequency-domain vector $\mathbf{S}$ to a time-domain OFDM symbol via the N_{FFT} -point IDFT:

$$x[n] = \frac{1}{\sqrt{N_{\text{FFT}}}} \sum_{k=0}^{N_{\text{FFT}}-1} S[k] e^{j2\pi kn/N_{\text{FFT}}}, \quad n = 0, \dots, N_{\text{FFT}} - 1.$$

Step 10: Cyclic Prefix Insertion. Prepend the last N_{CP} samples of $x[n]$ to its begin-

ning to produce the transmitted sequence of length $N_{\text{FFT}} + N_{\text{CP}}$:

$$\tilde{x}[n] = \begin{cases} x[N_{\text{FFT}} + n], & 0 \leq n < N_{\text{CP}}, \\ x[n - N_{\text{CP}}], & N_{\text{CP}} \leq n < N_{\text{FFT}} + N_{\text{CP}}. \end{cases}$$

The CP length must satisfy $N_{\text{CP}} \geq L - 1$.

Step 11: Multipath Channel Transmission. Convolve $\tilde{x}[n]$ with the L -tap channel $h[\ell]$ and add complex AWGN:

$$\tilde{y}[n] = \sum_{\ell=0}^{L-1} h[\ell] \tilde{x}[n - \ell] + w[n], \quad w[n] \sim \mathcal{CN}(0, \sigma_w^2).$$

Draw $h[\ell] \sim \mathcal{CN}(0, 1/L)$ independently for each OFDM symbol. Retain only the first $N_{\text{FFT}} + N_{\text{CP}}$ samples of the convolution output.

Channel Coding Schemes

All encoders share the same interface: they accept a code block of up to the maximum permitted size and return the coded bit sequence. All decoders accept a soft LLR vector and return decoded bits with a success/failure flag.

Scheme A — 5G NR LDPC Code (standards baseline)

Implement the 5G NR LDPC encoder and iterative decoder as specified in 3GPP TS 38.212, Section 5.3:

- Base graph selection.** Use BG1 for $A > 3824$ bits or $R \geq 0.67$, and BG2 otherwise, following the selection rule in TS 38.212, Section 5.2.2.
- Lifting size Z .** Select the smallest valid lifting size from the set defined in TS 38.212 such that $22Z \geq K$ (BG1) or $10Z \geq K$ (BG2), where K is the information block length including filler bits.
- Encoder.** Implement systematic encoding using the lifted parity-check matrix H constructed from the base matrix and circulant shifts. Filler bits must be set to zero before encoding and punctured from the output.
- Decoder.** Implement iterative belief propagation (BP) decoding or the min-sum approximation. The number of decoding iterations must be a configurable parameter (complexity knob). Apply early termination when all parity checks are satisfied.

Reference. The base matrices, lifting factor sets, and circulant shift tables for BG1 and BG2 are fully specified in 3GPP TS 38.212, Tables 5.3.2-2 through 5.3.2-5.

Scheme B — Terminated Convolutional Code

Implement a rate-1/2 convolutional code with zero-tail trellis termination and soft-decision Viterbi decoding:

- Generator polynomials.** The generator polynomial pair must be configurable in octal notation. Evaluate at least two constraint lengths, for example:

- $K = 7$: $(133, 171)_8$ — the industry-standard NASA polynomial pair used in 3G and WiMAX.
 - $K = 9$: $(561, 753)_8$ — longer constraint length for higher coding gain.
- (b) **Trellis termination.** After encoding the information bits, flush the shift register by appending $K - 1$ known zero bits (zero-tail termination). This forces the trellis to a known final state, enabling optimal Viterbi decoding, but introduces a rate overhead of $(K - 1)/n$ per code block that must be accounted for in all rate and SNR computations.
- (c) **Puncturing (optional).** To match a target code rate $R > 1/2$, implement a configurable puncturing pattern over the two output streams. Report the achieved rate and account for punctured positions as erased LLRs (set to zero) at the decoder.
- (d) **Viterbi decoder.** Implement soft-decision Viterbi decoding using the add-compare-select (ACS) recursion on path metrics. Hard-decision Viterbi is not acceptable. The decoder must perform a full traceback from the known terminal state.

Required. You must evaluate **at least two constraint lengths** and include separate BER/BLER curves for each, both with and without accounting for the termination overhead in the effective code rate.

PDSCH Receiver Chain

Implement the following steps in order.

1. **Cyclic Prefix Removal.** Discard the first N_{CP} samples of $\tilde{y}[n]$ to obtain the N_{FFT} -sample block:

$$y[n] = \tilde{y}[n + N_{\text{CP}}], \quad n = 0, \dots, N_{\text{FFT}} - 1.$$

Provided $N_{\text{CP}} \geq L - 1$, the CP converts the linear convolution with $h[\ell]$ into a circular convolution, so the frequency-domain relation $Y[k] = H[k]S[k] + W[k]$ holds exactly.

2. **N_{FFT} -Point DFT (OFDM Demodulation).** Transform the received block to the frequency domain:

$$Y[k] = \frac{1}{\sqrt{N_{\text{FFT}}}} \sum_{n=0}^{N_{\text{FFT}}-1} y[n] e^{-j2\pi kn/N_{\text{FFT}}}, \quad k = 0, \dots, N_{\text{FFT}} - 1,$$

where $H[k] = \sum_{\ell=0}^{L-1} h[\ell] e^{-j2\pi k\ell/N_{\text{FFT}}}$ is the channel frequency response on subcarrier k .

3. **Channel Equalization.** Using the known channel taps $h[\ell]$, compute $H[k]$ for each active subcarrier. Implement **both** of the following equalizers as separate, switchable functions and compare their performance (see Section 7):

Zero-Forcing (ZF) Equalizer. Invert the channel response exactly:

$$\hat{S}_{\text{ZF}}[k] = \frac{H^*[k]}{|H[k]|^2} Y[k] = S[k] + \frac{H^*[k]}{|H[k]|^2} W[k].$$

The post-equalization noise variance on subcarrier k is $\sigma_{ZF,k}^2 = \sigma_w^2/|H[k]|^2$, so the effective per-subcarrier SNR is

$$\gamma_{ZF,k} = \frac{|H[k]|^2}{\sigma_w^2}.$$

Minimum Mean-Square Error (MMSE) Equalizer. Regularise the inversion using the noise variance:

$$\hat{S}_{\text{MMSE}}[k] = \frac{H^*[k]}{|H[k]|^2 + \sigma_w^2} Y[k] = \mu_k S[k] + \tilde{W}_{\text{MMSE}}[k],$$

where the bias factor is $\mu_k = |H[k]|^2/(|H[k]|^2 + \sigma_w^2)$ and the post-equalization SNR is

$$\gamma_{\text{MMSE},k} = \frac{\mu_k |H[k]|^2}{\sigma_w^2}.$$

Note that $\mu_k \rightarrow 1$ at high SNR (MMSE $\rightarrow$ ZF) and $\mu_k \rightarrow 0$ at low SNR (MMSE attenuates rather than amplifies noise on deeply faded subcarriers).

LLR scaling is mandatory for both equalizers. The LLRs passed to the decoder must use the correct per-subcarrier SNR γ_k . A single global SNR is incorrect and will severely degrade performance in frequency-selective fading. For the MMSE equalizer, also account for the bias μ_k in the LLR computation; the equalised symbol is $\mu_k S[k]$ plus noise, not $S[k]$ plus noise.

4. **Subcarrier Demapping.** Extract the E/M equalised symbols $\hat{S}[k]$ from the active subcarrier positions, discarding DC and guard subcarriers.
5. **Demodulation (LLR Computation).** Compute soft LLRs for each coded bit from the equalised symbol $\hat{S}[k]$ using the per-subcarrier SNR γ_k . For QPSK, 16-QAM, and 64-QAM, use the exact or approximate max-log LLR expressions:

$$\lambda_b = \ln \frac{\sum_{s \in \mathcal{S}_b^{+1}} \exp(-\gamma_k |\hat{S}[k] - s|^2)}{\sum_{s \in \mathcal{S}_b^{-1}} \exp(-\gamma_k |\hat{S}[k] - s|^2)},$$

where $\mathcal{S}_b^{+1}$ and $\mathcal{S}_b^{-1}$ are the subsets of constellation points with bit b equal to +1 and −1 respectively. For QPSK only, this simplifies to the linear approximation used in the PDCCH assignment.

6. **Descrambling.** Flip the signs of the appropriate LLRs using the same pseudo-random sequence generated from the RNTI and cell ID as at the transmitter.
7. **Rate Recovery.** Restore the full coded LLR vector for each code block by re-inserting zero LLRs at punctured positions or $\pm\infty$ LLRs at shortened/repeated positions, reversing the rate-matching applied at the transmitter.
8. **Decoding.** Decode each code block independently:

- **LDPC:** run iterative BP or min-sum decoding for the configured number of iterations. Declare failure if parity checks are not satisfied after the maximum number of iterations.
 - **Convolutional:** run soft-decision Viterbi decoding with traceback to the known terminal state. The decoder always produces a hard decision; failure is detected only via the subsequent CRC check.
9. **Code Block Reassembly.** Concatenate the decoded bits from all code blocks, stripping the per-block CRC24B bits (if segmentation was applied). Verify the CRC24B on each block before reassembly; if any block fails its CRC, mark the transport block as failed immediately.
 10. **CRC Check.** Verify the transport block CRC24A over the reassembled payload. Declare a transport block error (BLER event) if the CRC fails.

ZF vs. MMSE Equalizer Comparison

Implement ZF and MMSE as separate modules with a common interface: both accept $Y[k]$, $H[k]$, and σ_w^2 and return $\hat{S}[k]$ and γ_k . This interface ensures the rest of the receiver chain is identical for both equalizers.

Required equalizer comparison plots

For each equalizer, using a fixed coding scheme (e.g. 5G LDPC at rate 1/2) and a fixed number of channel taps L , produce:

- **BER vs. E_b/N_0** for ZF and MMSE on the same axes, for at least two values of L (e.g. $L = 4$ and $L = 16$).
- **BLER vs. E_b/N_0** for ZF and MMSE on the same axes.
- **Per-subcarrier SNR distribution:** plot the empirical CDF of $\gamma_{ZF,k}$ and $\gamma_{MMSE,k}$ at a fixed E_b/N_0 (e.g. 5 dB) to illustrate how MMSE smooths out deep fades. Choose a representative channel realisation and overlay both distributions.

LLR mismatch experiment. As an additional diagnostic, simulate both equalizers *without* per-subcarrier LLR scaling (i.e. use a constant global SNR for all subcarriers) and compare the resulting BLER curves to the correctly scaled versions. This illustrates the importance of accurate LLR computation and is worth including in your report discussion.

Matched Complexity Constraint

Motivation

LDPC decoding complexity scales with the number of BP iterations and the code block size. Viterbi decoding complexity scales exponentially with the constraint length K (trellis has 2^{K-1} states) and linearly with blocklength. Comparing these schemes at a fixed iteration count or a fixed constraint length is not fair. All comparisons must be made at **equal average decoding time per transport block**.

Complexity control parameters

Scheme	Primary complexity knob	Secondary knob
5G NR LDPC	Number of BP iterations $I_{\max}$	Early-termination threshold
Convolutional ($K = 7$)	— (fixed trellis)	Block size per segment
Convolutional ($K = 9$)	— (fixed trellis)	Block size per segment

Note that the Viterbi decoder has fixed complexity for a given K and blocklength, whereas LDPC complexity is adjustable via iteration count. Sweep $I_{\max}$ for LDPC and select the value that gives the same average decoding time as the Viterbi decoder at a mid-range SNR point (e.g. $E_b/N_0 = 3$ dB).

Required complexity measurements

For each scheme and operating point, measure and report:

- **Mean decoding time** per transport block, averaged over at least 500 transport blocks at $E_b/N_0 = 3$ dB.
- **Standard deviation** of decoding time per transport block.
- **Approximate operation count:** for LDPC, count the number of check-node and variable-node messages processed per iteration; for Viterbi, count ACS operations per information bit.
- **Scaling with blocklength:** plot mean decoding time as a function of transport block size A for both schemes.

Performance Metrics and Required Plots

Error rate curves

For each configuration (A, E, R, M) and each coding scheme, using both ZF and MMSE equalizers, plot:

- **BER vs. E_b/N_0** over the range 0–8 dB (or until $\text{BER} < 10^{-5}$).
- **BLER vs. E_b/N_0** over the same range.

Collect enough Monte Carlo trials to obtain at least **100 transport block errors** at each simulated SNR point.

Effect of modulation order

For a fixed coding scheme (e.g. LDPC at rate $1/2$) and fixed equalizer, overlay BER/BLER curves for QPSK, 16-QAM, and 64-QAM on the same axes. This isolates the effect of spectral efficiency on error performance.

Complexity–performance tradeoff

Plot **BLER at a fixed E_b/N_0** (e.g. 4 dB) as a function of mean decoding time per transport block, with one point/curve per scheme and equalizer combination. This is the central comparison figure.

Convolutional code termination overhead

For each constraint length, plot BLER both with and without correcting the SNR axis for the termination rate overhead. This quantifies the practical penalty introduced by zero-tail termination at the simulated block sizes.

Required Analysis and Discussion

Your report must address the following questions with supporting simulation evidence:

1. **Matched-complexity comparison.** At equal average decoding complexity, which scheme achieves lower BLER? Does the answer depend on the transport block size A , modulation order M , or number of channel taps L ?
2. **ZF vs. MMSE equalizer.** At what SNR range does MMSE offer a meaningful improvement over ZF? How does the gap between the two equalizers change with L ? Explain your observations in terms of the per-subcarrier SNR distributions you plotted.
3. **Effect of frequency selectivity.** Compare BLER for $L = 1$ (flat fading), $L = 8$, and $L = 16$ (frequency selective). Does the coding gain from LDPC increase or decrease with L ? Why does multipath diversity interact differently with LDPC and convolutional codes?
4. **Modulation order and code rate interaction.** How does increasing the modulation order from QPSK to 64-QAM affect the required E_b/N_0 to achieve a target BLER of 10^{-2} ? Is the gap between LDPC and convolutional codes larger or smaller at higher modulation orders?
5. **Constraint length and termination overhead.** Quantify the BLER gain from increasing the constraint length from $K = 7$ to $K = 9$. At what block size does the termination overhead become negligible (less than 0.1 dB SNR penalty)?
6. **Iterative decoding behaviour.** Plot BLER vs. number of BP iterations for LDPC at a fixed SNR. At what iteration count does performance saturate? How does early termination affect average decoding time versus worst-case decoding time?

Deliverables

Code submission

Submit a single compressed archive containing:

- Well-structured, commented source code with a clear module hierarchy: `encoder/`, `decoder/`, `ofdm/`, `channel/`, `equalizer/`, `simulation/`.
- A `README` explaining how to reproduce every plot in the report, with exact command-line arguments or configuration files.
- Unit tests for each processing block: CRC attachment/checking, code block segmentation, LDPC encoder/decoder, convolutional encoder/Viterbi decoder, IDFT/DFT, CP insertion/removal, and both equalizers.

Simulation plots

Provide all plots in a single PDF. Each figure must be labelled with:

- Coding scheme, configuration (A, E, R, M, L), equalizer type, and complexity operating point.
- Axis labels with units; legend entries for all curves.
- Number of Monte Carlo transport blocks simulated per SNR point.

Final report

The report should be concise (8–14 pages, excluding plots and references) and must contain:

1. **Introduction:** motivation, problem statement, and scope.
2. **System model:** transmitter chain, channel model, and receiver chain with both equalizers described.
3. **Coding scheme descriptions:** LDPC base graph construction, BP decoding; convolutional encoder, trellis structure, Viterbi.
4. **Complexity matching methodology:** how decoding times were measured and matched.
5. **Results:** all required plots with captions.
6. **Analysis:** answers to the six questions in Section 9.
7. **Conclusion:** summary of findings and a recommendation for which scheme and equalizer combination to use under different operating conditions.

References

Standards

- 3GPP TS 38.212 v17: *NR; Multiplexing and Channel Coding*. (LDPC base graphs, lifting, segmentation, rate matching — Sections 5.1–5.4.)
- 3GPP TS 38.211 v17: *NR; Physical Channels and Modulation*. (Scrambling, modulation mapping, subcarrier spacing — Sections 5.1, 7.3.)
- 3GPP TS 38.214 v17: *NR; Physical Layer Procedures for Data*. (PDSCH rate selection, MCS tables — Section 5.1.)

Textbooks and research papers

- T. Richardson and R. Urbanke, *Modern Coding Theory*. Cambridge University Press, 2008. [*LDPC codes, density evolution, belief propagation*.]
- A. J. Viterbi, “Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm,” *IEEE Trans. Inf. Theory*, vol. 13, no. 2, 1967.

- J. G. Proakis, *Digital Communications*, 5th ed. McGraw-Hill, 2008. [*OFDM, equalization, LLR computation for QAM.*]
- S. Coleri et al., “Channel Estimation Techniques Based on Pilot Arrangement in OFDM Systems,” *IEEE Trans. Broadcast.*, vol. 48, no. 3, 2002. [*Useful background on frequency-domain equalization.*]

Tutorial resources

- ShareTechnote: 5G/NR — PDSCH.
https://www.sharetechnote.com/html/5G/5G_PDSCH.html
- ShareTechnote: 5G/NR — LDPC.
https://www.sharetechnote.com/html/5G/5G_ChannelCoding_LDPC.html